

Patrones y antipatrones aplicados a Hexagonal

Sección del documento técnico. El taller pide **seleccionar solo los patrones que el estilo realmente necesita** y justificar cuándo usarlos. Cada patrón aplicado enlaza al código; cada antipatrón evitado trae su evidencia, varias de ellas **verificadas por pruebas automáticas** (`test_arquitectura.py`).

Resumen

Patrón	Para qué lo necesita Hexagonal	Dónde está
Repository	Es el puerto de salida: el núcleo guarda y consulta sin saber qué base de datos hay detrás.	Puertos en <code>ports/repositories.py</code> ; implementación en <code>memory/repositories.py</code> (Mongo: próxima tarea).
Adapter	Es la mitad del nombre del estilo: traduce una tecnología a la forma que entiende el puerto.	Entrada: routers de <code>adapters/inbound/api/</code> . Salida: <code>InMemory*Repository</code> .
Dependency Injection	Sin inyección, el núcleo tendría que crear sus adaptadores y dependería de ellos.	<code>Depends(get_repositorios)</code> en cada router; el cableado vive solo en <code>dependencies.py</code> .
Facade (opcional, aplicado)	Da al adaptador de entrada una sola operación por acción de negocio, ocultando la orquestación de varios repositorios.	Cada caso de uso: <code>AprobarPostulacion.ejecutar()</code> coordina animal, postulaciones y cierre automático.
Anti-Corruption Layer (opcional, aplicado)	Impide que el modelo externo (JSON:API, documentos de Mongo) contamine el modelo del dominio.	Backend: <code>schemas.py</code> (<code>a_entidad()</code>) y <code>jsonapi.py</code> . Frontend: <code>normalizar()</code> en <code>jsonapi.js</code> .

Antipatrón	¿Se evita?	Evidencia
Anemic Domain Model	✅ (con matiz)	<code>Animal</code> y <code>Postulacion</code> tienen el comportamiento; lo verifica <code>test_las_entidades_con_reglas_no_son_anemicas</code> .
Tight Coupling	✅ en el núcleo	El dominio y la aplicación no importan infraestructura; lo verifica <code>test_regla_de_dependencia_hacia_adentro</code> .
Big Ball of Mud	✅	Capas explícitas con reglas de dependencia comprobadas por pruebas.
Fat Controllers	✅ (con matiz)	Los routers no cambian estados ni lanzan excepciones de dominio; lo verifica <code>test_los_routers_no_contienen_reglas_de_negocio</code> .
Overengineering	⚠️ Riesgo asumido y acotado	4 entidades con puertos y adaptadores; se mitigó no construyendo lo que no se necesita (ver abajo).

1. Patrones aplicados

Repository

Qué es: una colección de entidades en memoria, desde el punto de vista del núcleo, con operaciones como `obtener`, `guardar` y consultas con nombre del negocio.

En el proyecto: cuatro puertos (`AnimalRepository`, `PostulacionRepository`, `RefugioRepository`, `AdoptanteRepository`), definidos como `typing.Protocol`. Sus métodos hablan el lenguaje del dominio y no el de la base de datos: `existe_pendiente(adoptante_id, animal_id)` existe para la regla 2, y `listar_por_animal(animal_id, estado=PENDIENTE)` para el cierre automático.

```
class PostulacionRepository(Protocol):
    async def existe_pendiente(self, adoptante_id: str, animal_id: str) -> bool: ...
    async def listar_por_animal(self, animal_id: str, *, estado=None) -> list[Postulacion]: ...
    async def guardar(self, postulacion: Postulacion) -> Postulacion: ...
```

Cuándo usarlo: siempre que el núcleo deba persistir sin conocer el motor. **Cuándo no:** si la aplicación es un CRUD sin reglas, un ORM directo es suficiente.

Adapter

Qué es: convierte la interfaz de una tecnología en la interfaz que espera el cliente (el puerto o el caso de uso).

En el proyecto: - **Adaptadores de entrada (driving):** los routers de FastAPI (`animals.py`, `postulaciones.py`, `refugios.py`, `adoptantes.py`) convierten una petición JSON:API en una llamada a un caso de uso, y el resultado en un documento JSON:API. `errors.py` adapta las excepciones de dominio a códigos HTTP. - **Adaptadores de salida (driven):** `InMemoryAnimalRepository` y compañía implementan los puertos con diccionarios; el adaptador de Mongo los implementará con Motor. Ambos son intercambiables.

Cuándo usarlo: en cada frontera con una tecnología externa. Es la esencia del estilo.

Dependency Injection

Qué es: las dependencias se entregan desde fuera en vez de crearse dentro.

En el proyecto, en dos niveles: 1. **Por constructor en los casos de uso:** `Postular(animales, postulaciones, adoptantes)` recibe sus puertos; no sabe cuál implementación le llega. 2. **Con `Depends()` de FastAPI en los routers:** `python async def postular(cuerpo: PostulacionCrear, repos: Repositorios = Depends(get_repositorios)):` `postulacion = await Postular(repos.animales, repos.postulaciones, repos.adoptantes).ejecutar(...)` `get_repositorios()` está en `dependencias.py`, el **composition root**: el único archivo que conoce las implementaciones concretas. Las pruebas lo reemplazan con `app.dependency_overrides[get_repositorios]`, y así la app completa corre sin Mongo.

Cuándo usarlo: siempre en Hexagonal; es lo que materializa la inversión de dependencias. Se prefirió `Depends()` a un contenedor de DI externo (YAGNI).

Facade (opcional, aplicado)

Cada caso de uso es una fachada de la capa de aplicación: el router llama a **una** operación (`AprobarPostulacion.ejecutar(id)`) y no conoce la coreografía interna (buscar postulación y animal, aprobar, adoptar, guardar en orden y cerrar las demás pendientes). El resultado se devuelve en un solo objeto (`ResultadoResolucion`), con lo que el router arma la respuesta sin volver a consultar.

Cuándo usarlo: cuando una acción de negocio toca varios repositorios o entidades. **Cuándo no:** para lecturas triviales, aunque aquí se mantuvo por uniformidad (`ConsultarAnimal`).

Anti-Corruption Layer (opcional, aplicado)

Hay tres modelos distintos del mismo concepto y nunca se mezclan:

Modelo	Ejemplo para "animal"	Traductor
JSON:API (externo, camelCase)	<code>edadMeses</code> , <code>detallesEspecie</code> , <code>fotos:</code> <code>["url"]</code>	<code>schemas.py</code> → <code>a_entidad()</code> ; <code>jsonapi.py</code> → <code>animal()</code>
Dominio (interno)	<code>Animal(edad_meses, detalles_especie,</code> <code>fotos=[Foto])</code>	—
Documento Mongo (semilla)	<code>edad_meses</code> , colección <code>animales</code> , <code>motivo_rechazo</code> libre	Adaptador de Mongo (contrato, sección 1)

En el frontend, `normalizar()` (`jsonapi.js`) cumple el mismo papel: las vistas reciben siempre la misma forma aunque el backend serialice distinto.

Cuándo usarlo: cuando un modelo externo tiene nombres, formas o semánticas distintas del dominio (aquí: camelCase frente a snake_case, fotos como URL frente a objeto, texto libre frente a enum).

Patrones que se decidió no usar

Patrón	Por qué no (por ahora)
Unit of Work / transacciones	El MongoDB del <code>docker-compose</code> es un nodo único (sin <i>replica set</i>); el orden de escritura al aprobar mitiga fallas a mitad.
CQRS	Lecturas y escrituras son simples y comparten modelo; separarlas duplicaría código.
Domain Events	El cierre automático ocurre en la misma operación; no hay otros suscriptores (correo, notificaciones).
Factory / Specification	Pydantic construye y valida las entidades; las consultas son pocas y con nombre propio en el puerto.
Contenedor de DI externo	<code>Depends()</code> de FastAPI basta.

2. Antipatrones que el diseño evita

Anemic Domain Model

El antipatrón: entidades que son solo datos (getters y setters) y la lógica repartida en servicios o controladores.

Cómo se evita: las reglas viven en las entidades que las poseen.

```
animal.validar_postulable()      # regla 1: el animal sabe si puede recibir postulaciones
animal.marcado_adoptado()        # solo desde "postulado"; si no, TransicionInvalida
postulacion.aprobar()            # solo si está pendiente; si no, PostulacionYaResuelta
postulacion.cerrar_automáticamente()
```

Matiz honesto: `Refugio` y `Adoptante` sí son solo datos, porque no tienen reglas propias en este dominio. Darles métodos artificiales sería overengineering. Y la regla 2 (duplicada) está en el caso de uso, no en una entidad, porque necesita consultar otras postulaciones a través del repositorio.

Tight Coupling

El antipatrón: módulos que dependen de implementaciones concretas, de modo que cambiar uno obliga a cambiar los demás.

Cómo se evita: el núcleo depende de `Protocol`, nunca de clases concretas. Las pruebas de arquitectura lo verifican sobre los imports reales: - `domain` no importa FastAPI, Motor, pymongo ni otras capas. - `application` no importa infraestructura ni adaptadores. - Los routers no importan adaptadores de salida (solo `dependencies.py` lo hace). - Los adaptadores de salida no conocen HTTP.

Excepción conocida (periferia): `InMemoryPostulacionRepository` recibe el `InMemoryAnimalRepository` concreto para filtrar por refugio. Es aceptable en un adaptador de pruebas; en Mongo se resolverá con una consulta.

Big Ball of Mud

El antipatrón: código sin estructura reconocible, donde todo depende de todo.

Cómo se evita: la estructura del repositorio es la del estilo, y las fronteras están comprobadas por pruebas, no solo por convención:

<code>app/domain/</code>	189 líneas	entidades, reglas y excepciones
<code>app/application/</code>	356 líneas	puertos y casos de uso
<code>app/adapters/inbound/api/</code>		routers, JSON:API, errores, cableado
<code>app/adapters/outbound/persistence/</code>		memory/ (y mongo/)

Fat Controllers

El antipatrón: controladores que validan, deciden reglas de negocio, acceden a la base de datos y arman la respuesta, todo junto.

Cómo se evita: cada endpoint hace tres cosas: recibe el documento ya validado por Pydantic, llama a **un** caso de uso y serializa el resultado. `test_los_routers_no_contienen_reglas_de_negocio` falla si un router importa excepciones de dominio, asigna un estado o llama métodos de reglas como `aprobar()` o `existe_pendiente()`.

Matiz honesto: `GET /postulaciones` intersecta los filtros por adoptante y por refugio dentro del router, y los `include` se resuelven consultando repositorios desde el adaptador. Es lógica de consulta y

presentación, no de negocio; si crece, conviene moverla a `ListarPostulaciones`.

Overengineering (riesgo propio de Hexagonal)

El riesgo: para un dominio pequeño, puertos, adaptadores, mapeos y casos de uso pueden costar más de lo que aportan.

Cómo se reconoce en el proyecto: 4 entidades, 4 puertos, 11 casos de uso y $\approx$ 1500 líneas de backend, de las cuales el 61 % son adaptadores. Un CRUD en capas sería más corto.

Cómo se acotó: - Se construyó solo lo necesario: sin Unit of Work, CQRS, eventos ni contenedor de DI (ver la tabla anterior). - Los puertos declaran solo los métodos que algún caso de uso usa (ningún `borrar()`). - La estructura se justifica por reglas de estado no triviales (postulación única activa y cierre automático) y se paga con evidencia: 100 % de cobertura del núcleo sin red ([evidencia de testabilidad](#)).

Conclusión: el riesgo existe y lo asumimos conscientemente; en un sistema sin la regla de cierre automático recomendaríamos un estilo en capas más simple.

Evidencia automática

Las afirmaciones marcadas como verificadas se comprueban en cada corrida de la suite:

Prueba (<code>test_arquitectura.py</code>)	Antipatrón que vigila
<code>test_regla_de_dependencia_hacia_adentro</code> (dominio y aplicación)	Tight Coupling, Big Ball of Mud
<code>test_solo_el_composition_root_conoce_los_adaptadores_de_salida</code>	Tight Coupling
<code>test_los_adaptadores_de_salida_no_conocen_http</code>	Big Ball of Mud
<code>test_los_routers_no_contienen_reglas_de_negocio</code>	Fat Controllers
<code>test_las_entidades_con_reglas_no_son_anemicas</code>	Anemic Domain Model

Fuentes

- Evans, E. (2003). *Domain-Driven Design*. Addison-Wesley (Repository, Anti-Corruption Layer).
- Fowler, M. (2002). *Patterns of Enterprise Application Architecture*. Addison-Wesley (Repository, Service Layer). Fowler, M. (2003). *AnemicDomainModel*. martinowler.com.
- Gamma, E. et al. (1994). *Design Patterns*. Addison-Wesley (Adapter, Facade).
- Foote, B., Yoder, J. (1997). *Big Ball of Mud*. PLoP.
- Cockburn, A. (2005). *Hexagonal Architecture (Ports and Adapters)*.